

Reviewer Pack — Minimal Rank of Symmetric Matrices over Finite Fields Bounds...

Entry 796fca78b143 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 06:14:33 UTC

Minimal Rank of Symmetric Matrices over Finite Fields Bounds Resolution Proof Width

Entry ID: 796fca78b143 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-31 06:14:33 UTC

1. Conjecture

Field A (mathematical branch): Linear Algebra over Finite Fields **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every Tseitin formula φ_G with n variables, the minimal rank of the symmetric matrices over the finite field associated with the induced incidence matrix of φ_G is linearly correlated with its resolution proof width $w(\varphi_G)$, such that $\text{minimal_rank}(M) = \Theta(w(\varphi_G))$

Rationale (proposer's reasoning):

The minimal rank of a matrix can serve as a measure of its complexity, and in linear algebra over finite fields, this measure is well-studied. If the rank of the incidence matrix of a Tseitin formula correlates with its resolution proof width, it may reveal a new structural link between linear algebra and the complexity of resolution proofs.

Taxonomy category: LINEAR_ALGEBRA_FINITE_FIELDS (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 60fd46ef23f0d798

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for at least 90% of the generated Tseitin formulas φ_G with n variables, the ratio of the minimal rank (over $\text{GF}(2)$) to the resolution proof width $w(\varphi_G)$ falls within the range $[0.5, 1.5]$, and no seed produces a ratio outside this range.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	1.00	SAFE	SAFE
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (2): - "minimal rank" AND "symmetric matrices" AND finite fields" - resolution proof complexity AND Tseitin formula AND minimal rank symmetric matrices finite fields

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
from fractions import Fraction

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    rank = 0
    for i in range(n):
        pivot = None
        for j in range(rank, m):
            if A[j][i] != 0:
                pivot = j
                break
        if pivot is None:
            continue
        A[pivot], A[rank] = A[rank], A[pivot]
        for j in range(m):
            if j != rank and A[j][i] != 0:
                factor = Fraction(A[j][i], A[rank][i])
                for k in range(n):
                    A[j][k] -= factor * A[rank][k]
        rank += 1
    return rank

def min_rank(M):
    M_copy = [row[:] for row in M]
    return gaussian_elimination(M_copy)

def tseitin_formula(n):
    variables = list(range(1, n + 1))
    clauses = []
    for i in range(2, n + 1):
        clauses.append([variables[i - 1], -variables[i - 2]])
        for j in range(i - 3, -1, -1):
            new_var = len(variables) + 1
```

```

        variables.append(new_var)
        clauses.append([-new_var, variables[j], -i])
        clauses.append([new_var, i])
    return variables, clauses

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    instances_tested = 0
    n_max = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        variables, clauses = tseitin_formula(n)
        M = [[0] * len(variables) for _ in range(len(clauses))]
        for j, clause in enumerate(clauses):
            for var in clause:
                if var > 0:
                    M[j][var - 1] = 1
                else:
                    M[j][-var - 1] = 1

        rank = min_rank(M)
        w_phi_G = len(clauses) # Simplified resolution proof width for Tseitin formulas

        if instances_tested == 0 or n > n_max:
            n_max = n
            instances_tested += len(clauses)

        ratio = Fraction(rank, w_phi_G)
        total_metric_value += ratio

        if not (Fraction(1, 2) <= ratio <= Fraction(3, 2)):
            conjecture_holds = False
            counterexample = f"n={n}, rank={rank}, w(φG)={w_phi_G}, ratio={ratio}"

    mean_metric_value = total_metric_value / instances_tested
    support_fraction = Fraction(instances_tested - sum(1 for _ in range(n_max) if not conjecture_holds),
                                instances_tested)

    return {
        "metric_name": "Ratio of Minimal Rank to Resolution Proof Width",
        "metric_value": float(mean_metric_value),
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = [run_trial(seed) for seed in seeds]

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)

```

```

support_fraction = Fraction(sum(1 for result in results if result["conjecture_holds"]), len(re

print("TRIALS:")
for result in results:
    print(f" TRIAL: {result}")

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0 support_fraction=1")
elif support_fraction >= Fraction(9, 10):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0 support_fraction={support_fraction}")
else:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete its execution to verify the conjecture. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	19623
2	preregistration	ollama_remote	glm4:latest	0	0	12347
3	novelty	ollama_remote	glm4:latest	0	0	13719
4	novelty	ollama_remote	glm4:latest	0	0	12537
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19915
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	29517
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16462
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13040
9	judge	ollama_remote	glm4:latest	0	0	8964

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 146126 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/796fca78b143.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/796fca78b143.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/796fca78b143.tar.gz (if generated)